Kernel Regression Assignment

Miguel García Moreno, Hugo Montilla Anta, Lola Velasco Ballesta

2026-10-20

Contents

Variables **1**

Task 1 **2**

Task 2 **5**

Task 3 **10**

Variables

We prepare the table by renaming the columns for convenience.

```
data <- read.table("auto-mpg.data")
colnames(data) <- c("mpg","cylinders","displacement","horsepower",
                    "weight","acceleration","model year",
                    "origin","car name")
```

We define the variables as requested. Careful!: x1 has `NA` values, so a mask is created to select those values and later be able to redefine x1 without those elements.

```
y <- data$mpg
x1 <- as.numeric(data$horsepower)
```

```
## Warning: NAs introducidos por coerción
```

```
x2 <- data$weight

mask=is.na(x1)
x1<- x1[!mask]
```

Once the variables are prepared we can move on to the exercises.

Task 1

Exercise 1. Determine the bandwidth parameter for the density function estimation of y, using a Gaussian kernel, with the following methods: cross-validation and plug-in.

To determine the bandwidth values we will use the following functions, which use the Gaussian kernel by default.

```
hvc_y=bw.ucv(y) #  Cross-validation
hvc_y
```

```
## [1] 1.797839
```

```
hpi_y=bw.SJ(y)  #  Plug-in
hpi_y
```

```
## [1] 1.94071
```

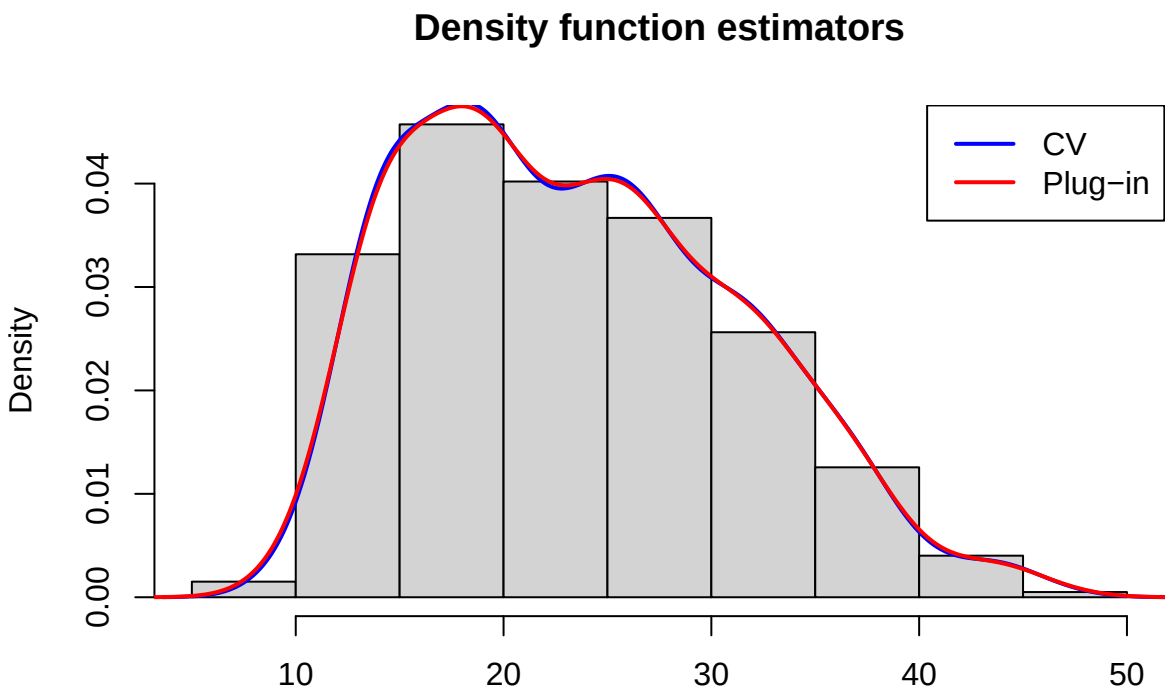
Exercise 2. Plot the associated kernel estimators on the same graph. Briefly comment on the result.

We define the different estimators, determining the bandwidths that should be used.

```
d1_y=density(y, bw = hvc_y) # Cross-validation
d2_y=density(y, bw = hpi_y) # Plug-in
```

We plot the estimators over the frequency histogram of variable y.

```
hist(y,freq=FALSE,main='Density function estimators', xlab='')
lines(d1_y, col='blue', lwd= 2 )
lines(d2_y, col = 'red',lwd= 2)
legend("topright", legend = c("CV", "Plug-in"), col = c("blue","red"),lwd = 2)
```



We can see that both types of estimates give very similar results. Comparing it with the real frequency histogram, we confirm that it seems to be a good fit.

Exercise 3. Repeat the two previous sections for x1 and x2.

x1

```
#x1
hvc_x1=bw.ucv(x1) #  Cross-validation
```

```
## Warning in bw.ucv(x1): minimum occurred at one end of the range
```

```
hvc_x1
```

```
## [1] 1.387066
```

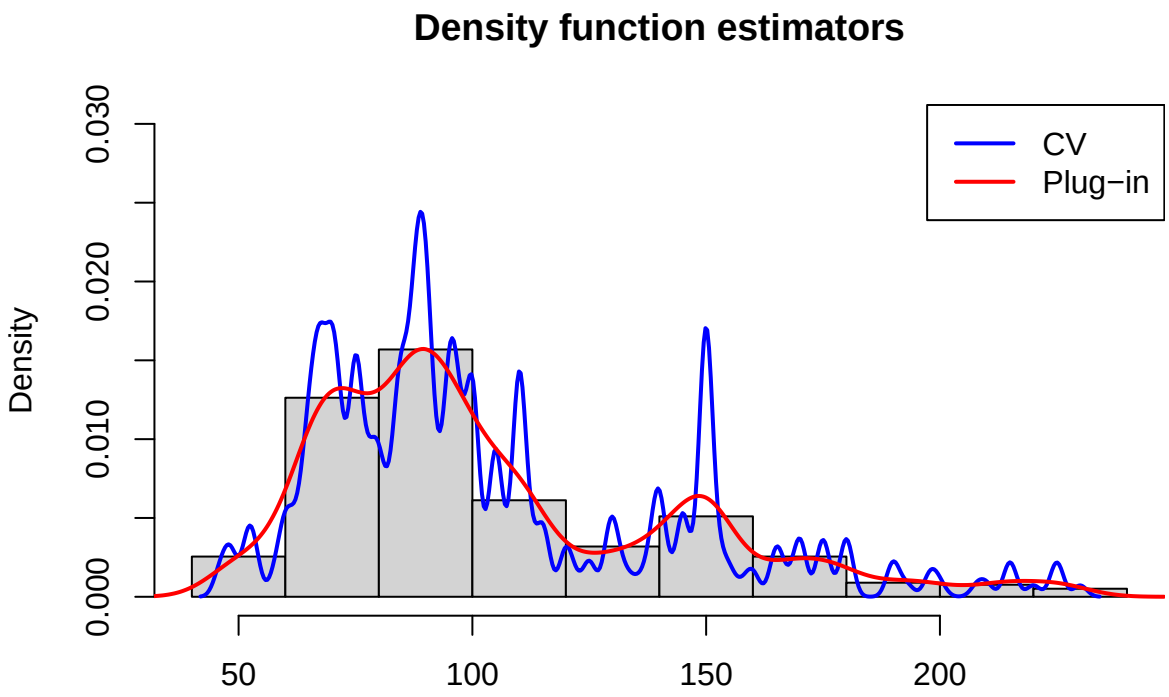
```
hpi_x1=bw.SJ(x1)  #  Plug-in
hpi_x1
```

```
## [1] 5.974012
```

```
d1_x1=density(x1, bw = hvc_x1) # Cross-validation
d2_x1=density(x1, bw = hpi_x1) # Plug-in

hist(x1,freq=FALSE,main='Density function estimators', xlab='', ylim=c(0,0.03))
```

```
lines(d1_x1, col='blue', lwd= 2 )
lines(d2_x1, col = 'red',lwd= 2)
legend("topright", legend = c("CV", "Plug-in"), col = c("blue","red"),lwd = 2)
```



Here we observe differences between the approximations. The LSCV estimate is rougher, with more folds, whereas the plug-in estimate is very smooth. This is because the plug-in bandwidth is noticeably larger than the LSCV one. This differs from the previous example, because in the previous exercise the bandwidths were very similar.

x2

```
#x2
hvc_x2=bw.ucv(x2) #  Cross-validation
hvc_x2
```

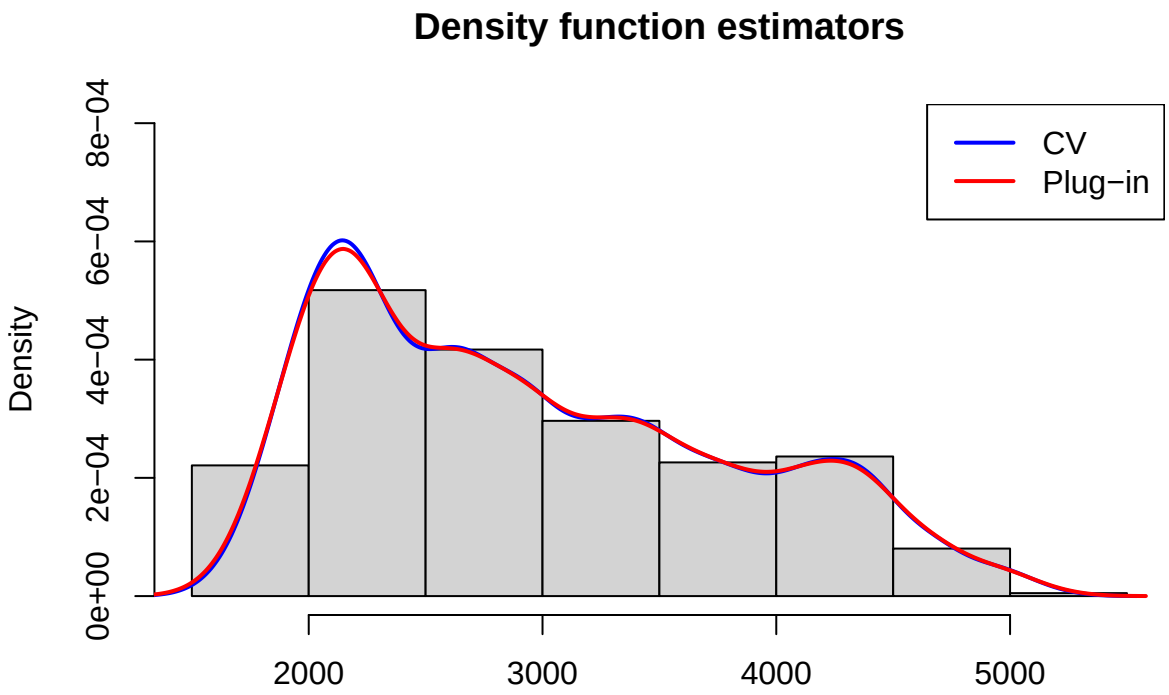
```
## [1] 134.0282
```

```
hpi_x2=bw.SJ(x2)  #  Plug-in
hpi_x2
```

```
## [1] 146.9706
```

```
d1_x2=density(x2, bw = hvc_x2) # Cross-validation
d2_x2=density(x2, bw = hpi_x2) # Plug-in
```

```r
hist(x2,freq=FALSE,main='Density function estimators', xlab='', ylim=c(0,0.0008))
lines(d1_x2, col='blue', lwd= 2 )
lines(d2_x2, col = 'red',lwd= 2)
legend("topright", legend = c("CV", "Plug-in"), col = c("blue","red"),lwd = 2)
```



In this graph we observe a behavior very similar to that of the first section.

Task 2

Exercise 4. Estimate the joint density function of (y, x1) using a kernel estimator, with a Gaussian kernel and bandwidth calculated with the plug-in method. Write the bandwidth matrix and plot the obtained estimator.

Careful!: We previously removed the `NA` elements from x1. Therefore, we must remove the corresponding ones from y.

```r
y_mask = y[!mask]

length(x1)
```

```
## [1] 392
```

```
length(y_mask)
```

```
## [1] 392
```

For the joint density function we will use the `ks` library.

```
library(ks)
```

We compute the bandwidth matrix.

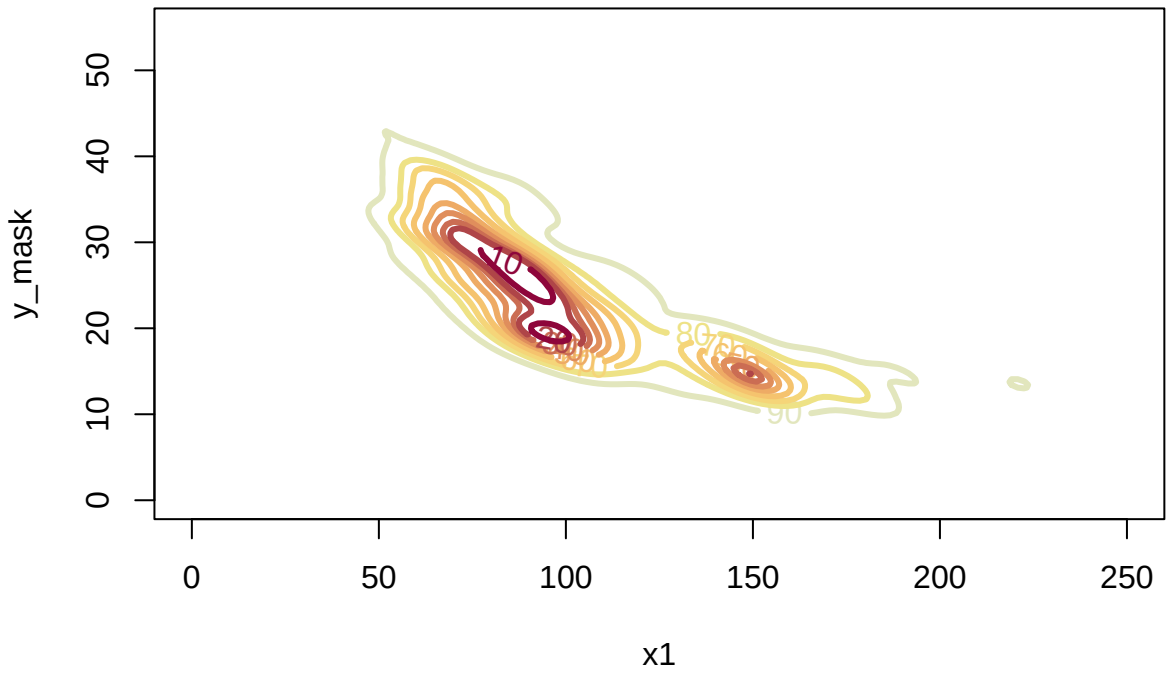
```
mat = cbind(x1,y_mask)
Hp = Hpi(mat)
Hp
```

```
##           [,1]       [,2]
## [1,]  66.10216 -10.980567
## [2,] -10.98057   3.695583
```

We express the joint density function using `kde`; if neither the bandwidth matrix nor the kernel are specified, it uses the plug-in method and the Gaussian kernel by default. We plot the contours.

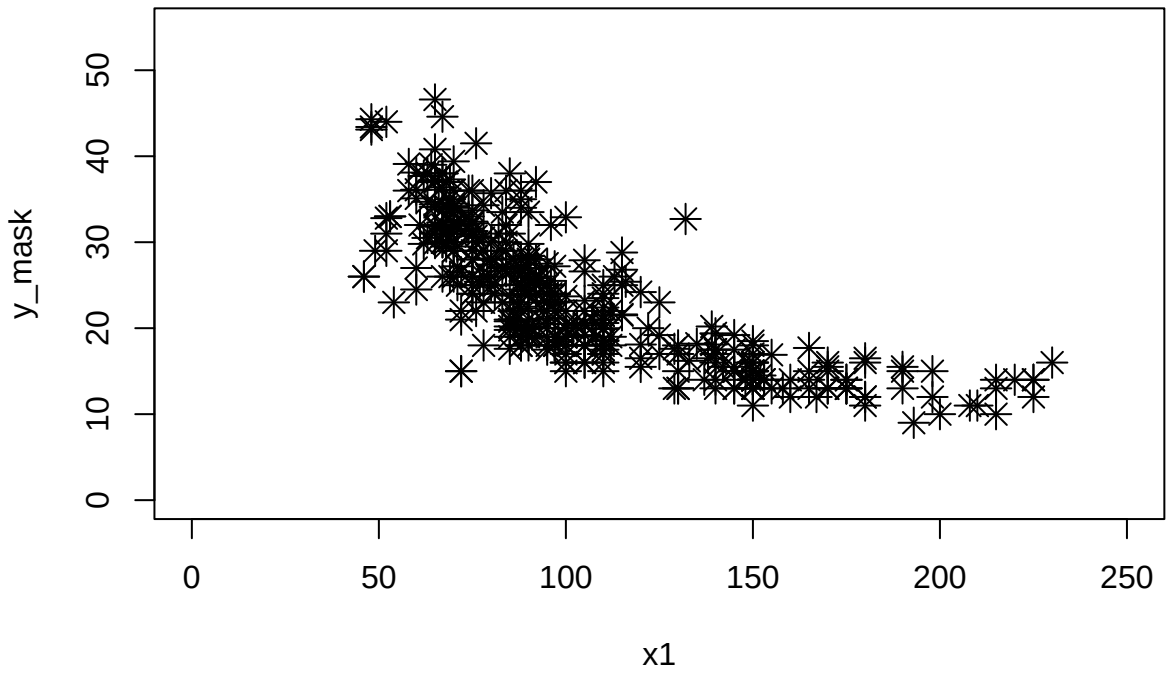
```
fhat.pi = kde(mat,H=Hp)

v= 1:9*10
plot(fhat.pi, cont=v, lwd=3,ylim=c(0,55),xlim=c(0,250)) # add contours
```



We compare it with the distribution.

```
plot(x1,y_mask, pch=8, cex=1.5,ylim=c(0,55),xlim=c(0,250))
```



We can see that they are very similar. It looks like a good estimate.

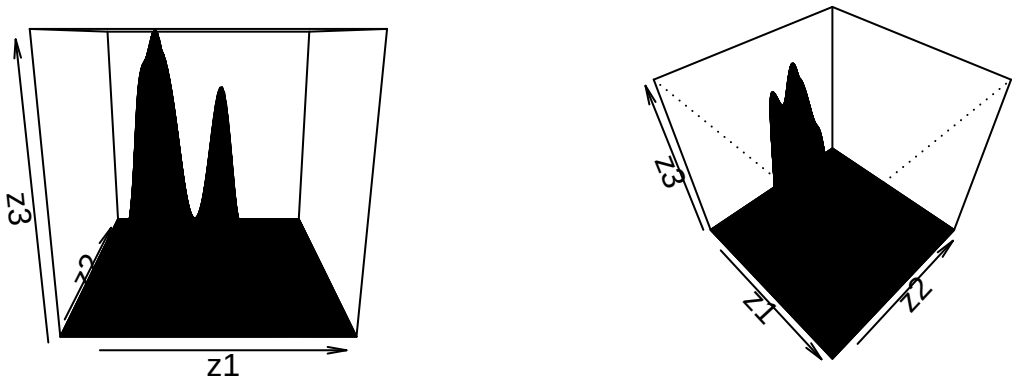
```
# 3D representation

z1 = fhat.pi$eval.points[[1]]

z2 = fhat.pi$eval.points[[2]]

z3 = fhat.pi$estimate

layout(matrix(1:2, ncol=2))
persp(z1,z2,z3,col='gold')
persp(z1,z2,z3,col='gold',theta=45,phi=45) # Same plot but rotated
```




```
layout(1) #so a single plot is shown again
```

Exercise 5. Give an estimate of the joint density function at (y, x1) = (20, 150).

```
predict(fhat.pi,x=c(150,20))
```

```
## [1] 8.706612e-05
```

Exercise 6. Repeat the two previous sections for (y, x2) and (y, x2) = (25, 3000).

Note: in this case we use the original y variable.

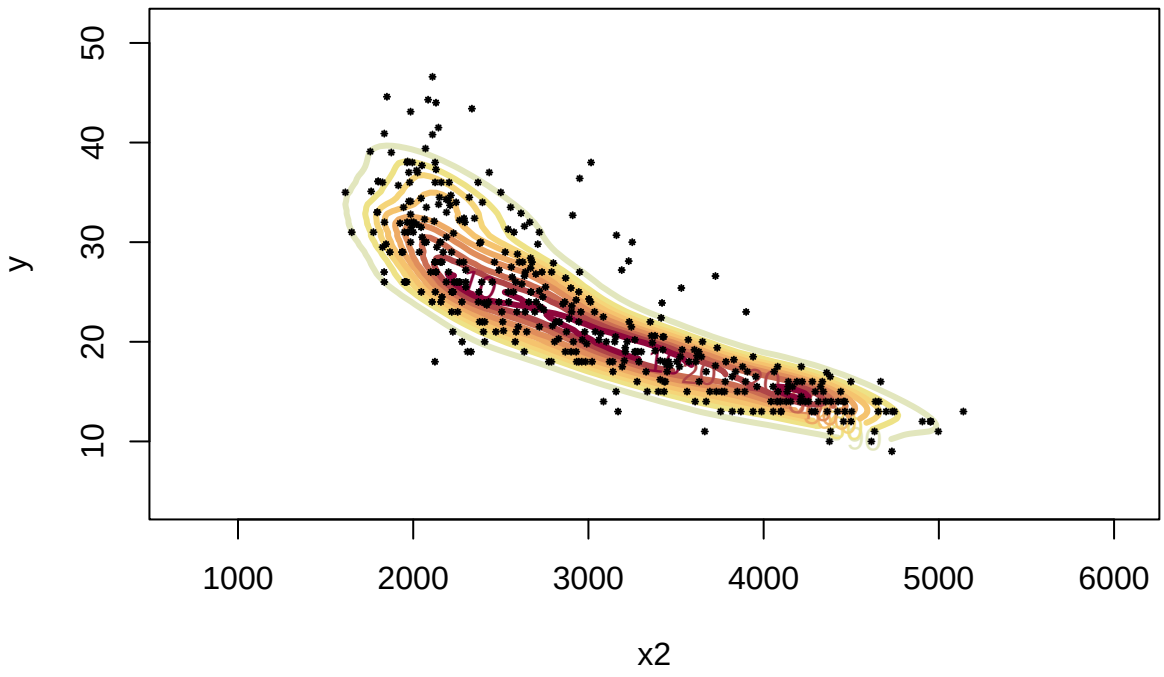
```
mat = cbind(x2, y)

# Bandwidth
Hp = Hpi(mat)
Hp
```

```
##             [,1]        [,2]
## [1,] 59829.8738 -387.091432
## [2,]  -387.0914    4.256489
```

```r
fhat.pi = kde(mat,H=Hp)

v= 1:9*10
plot(fhat.pi, cont=v, lwd=3)
points(x2, y,pch=8, cex=0.3)
```



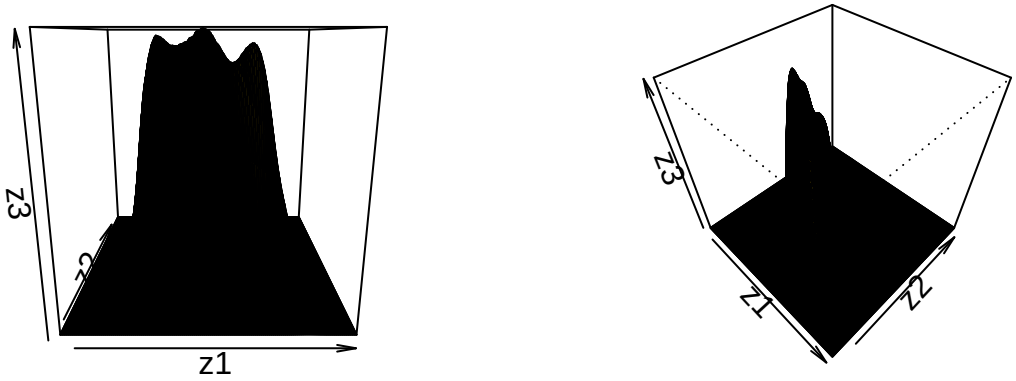

```r
# 3D representation

z1 = fhat.pi$eval.points[[1]]

z2 = fhat.pi$eval.points[[2]]

z3 = fhat.pi$estimate

layout(matrix(1:2, ncol=2))
persp(z1,z2,z3,col='gold')
persp(z1,z2,z3,col='gold',theta=45,phi=45) # Same plot but rotated
```




```
layout(1)
```

We compute the prediction of the joint density given the values stated in the problem.

```
predict(fhat.pi,x=c(3000,25))
```

```
## [1] 1.92177e-05
```

Task 3

Exercise 7. Estimate the regression curve of y on x1 using a local linear estimator (p = 1), with a Gaussian kernel, and bandwidth calculated with the following methods: cross-validation and plug-in.

We use locpoly from the `KernSmooth` package, which uses the Gaussian kernel and the bandwidth obtained by the 'plug-in' method by default.

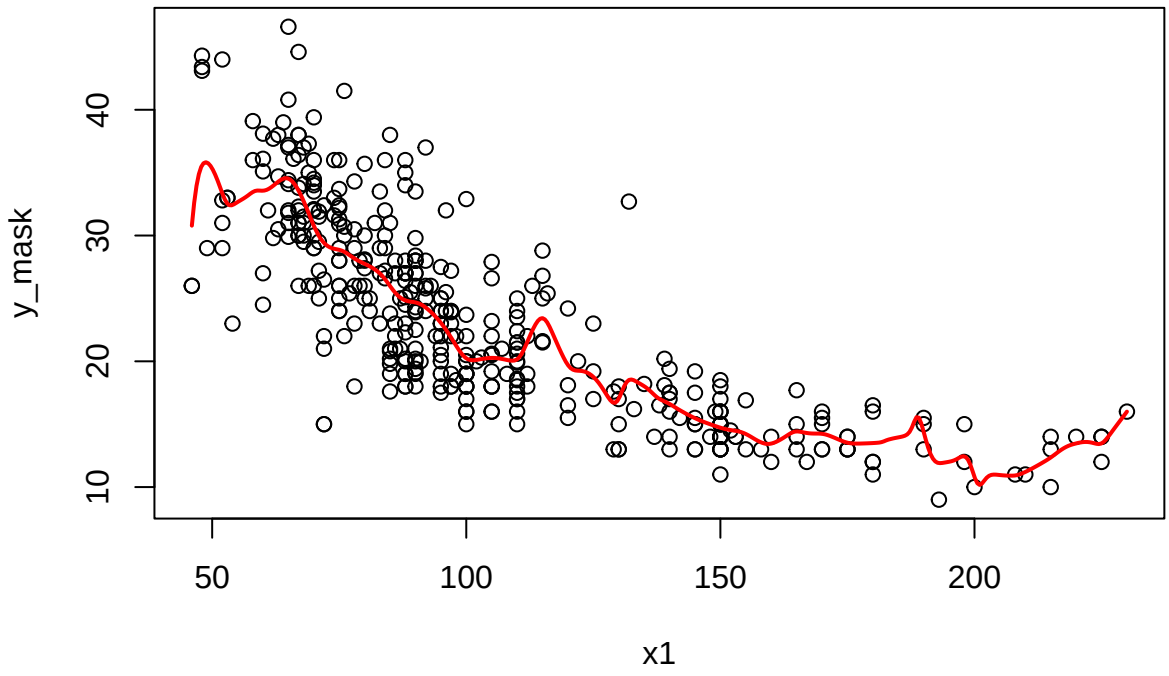
```
library(KernSmooth)
```

```
h_pi = dpill(x1,y_mask)
h_pi #plug-in bandwidth
```

```
## [1] 2.504173
```

```
m_pi = locpoly(x1, y_mask,bandwidth=h_pi)

plot(x1, y_mask)
lines(m_pi, col='red', lwd=2)
```



We use npregbw from the **np** package to compute the bandwidth via cross-validation.

```
library(np)
```

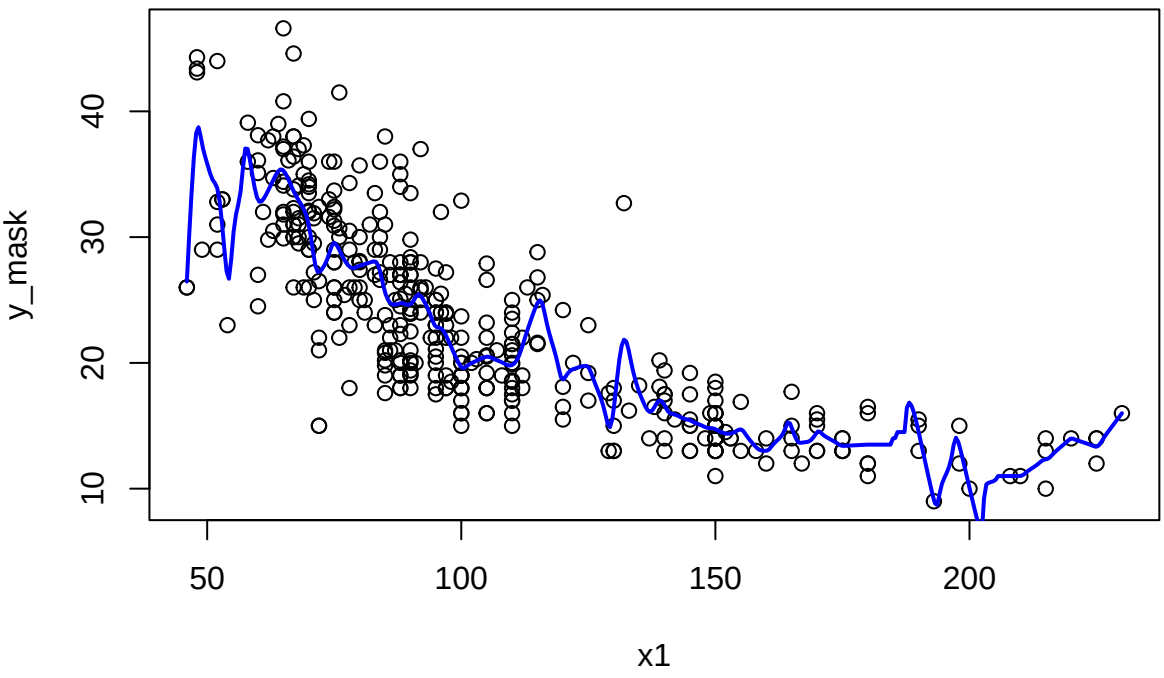
```
bw_vc = npregbw(y_mask~x1, regtype="ll") #the degree must be indicated
```

```
## Multistart 1 of 1 |Multistart 1 of 1 |Multistart 1 of 1 |Multistart 1 of 1 /Multistart 1 of 1 |Multis
```

```
h_vc = bw_vc$bw #cross-validation bandwidth

m_vc = locpoly(x1,y_mask, bandwidth = h_vc)

plot(x1, y_mask)
lines(m_vc, col="blue", lwd=2,type="l")
```



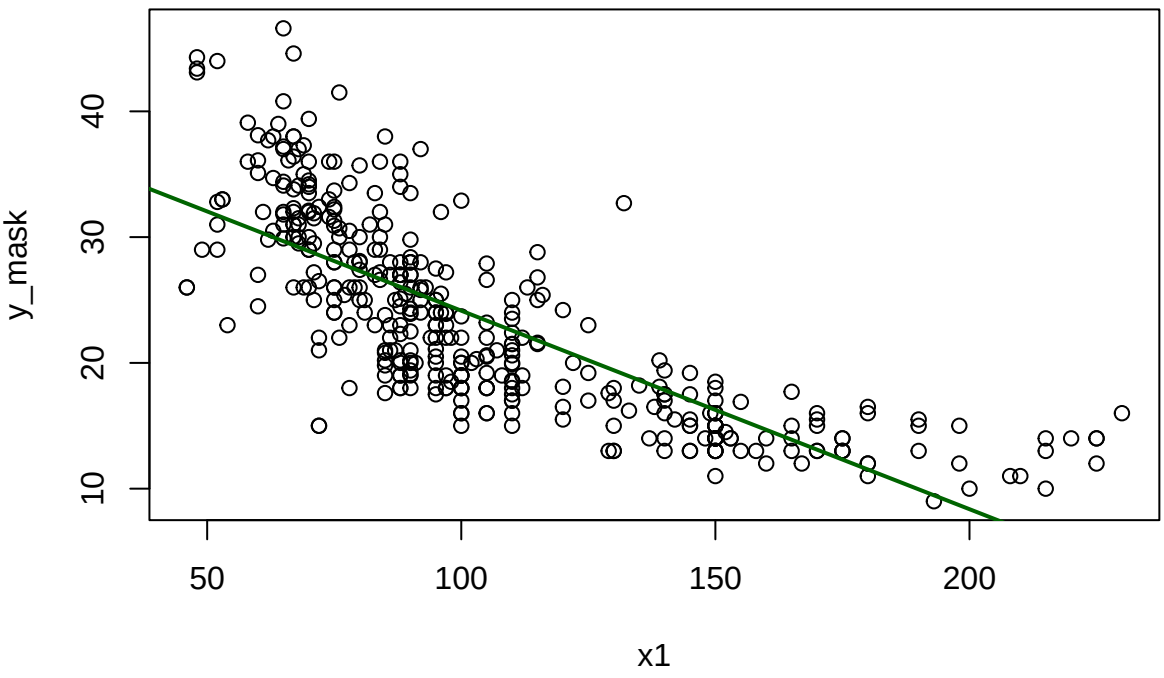
Looking at the plots we can see that the first method (plug-in) has a larger bandwidth, drawing a smoother estimated density function than the second method (cross-validation), which again has a smaller bandwidth.

Exercise 8. Compute the regression line and write its expression.

```
lr = lm(y_mask~x1)
lr
```

```
##
## Call:
## lm(formula = y_mask ~ x1)
##
## Coefficients:
## (Intercept)           x1
##     39.9359      -0.1578
```

```
plot(x1,y_mask)
abline(lr, col="darkgreen", lwd=2)
```

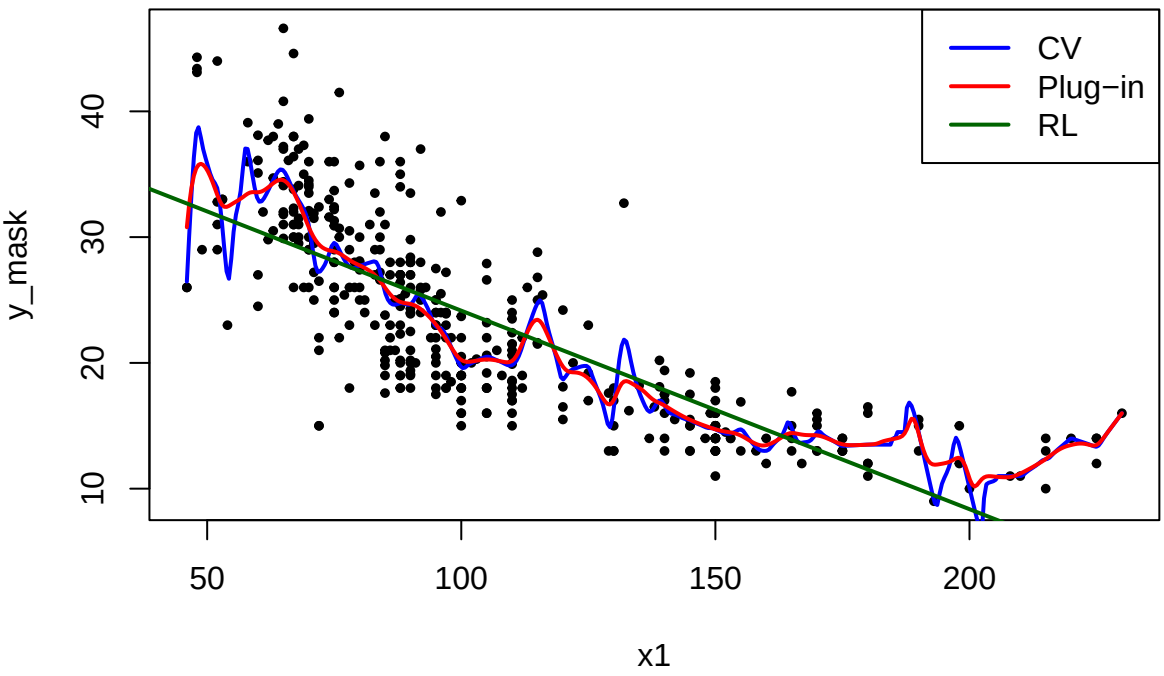


Its expression would be:

$y_mask = 39.9359 - 0.1578x_1$

Exercise 9. Plot the three estimators on the same graph. Briefly comment on the result.

```
plot(x1,y_mask,pch=20,cex=0.8)
lines(m_vc,col="blue",lwd=2)
lines(m_pi,col="red",lwd=2)
abline(lr,col="darkgreen",lwd=2)
legend("topright", legend = c("CV", "Plug-in","RL"), col = c("blue","red","darkgreen"),
       lwd = 2)
```



Plug-in is again a smoother estimate than the CV estimate. In this case it would probably be a good idea to use the plug-in method because it seems to adapt better to the variance of the data.

Exercise 10. Estimate the regression curve at x1 = 110 with the three estimators.

The locpoly function does not have a predict function, so we have to use approx.

```
pred_pi <- approx(x = m_pi$x,y = m_pi$y,
                  xout = 110)$y
pred_cv <- approx(x = m_vc$x,y = m_vc$y,
                  xout = 110)$y
pred_reg <- predict(lr,newdata=data.frame(x1=110))

predictions <- c(pred_pi,pred_cv,pred_reg)
names(predictions) <- c("Plug in","CrossValidation","RL")
predictions
```

```
##         Plug in CrossValidation             RL
##        20.16253        19.83646        22.57294
```

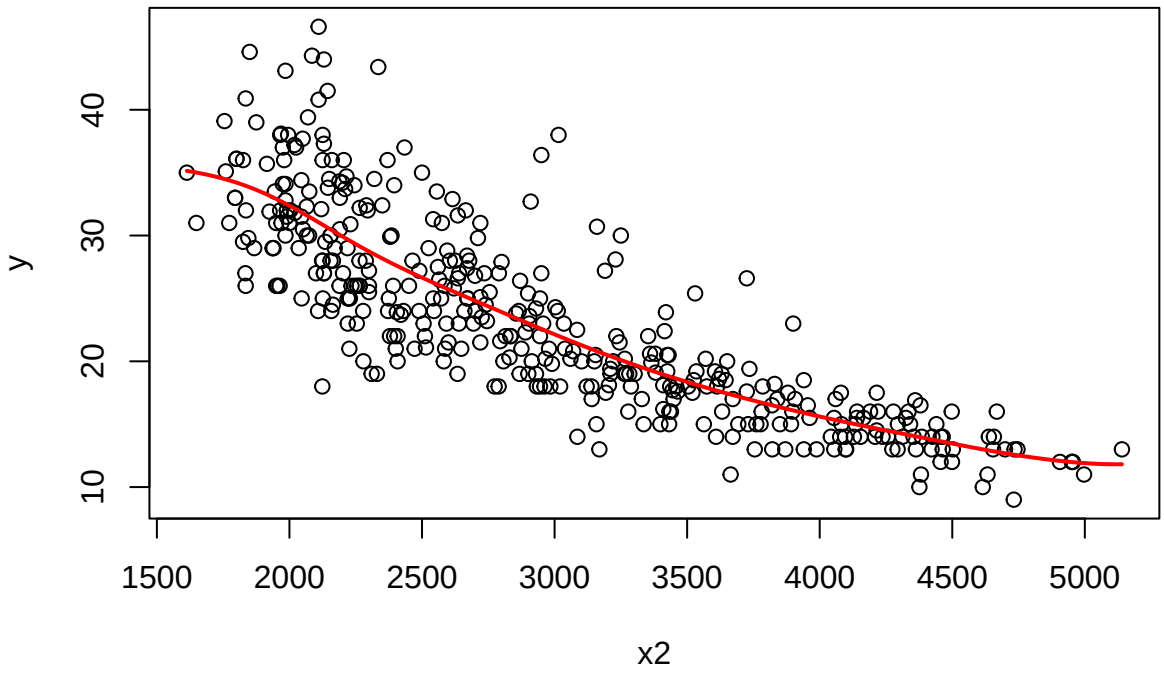
Exercise 11. Repeat the four previous sections for y on x2 and x2 = 3000.

```
h_pi = dpill(x2,y)
h_pi #plug-in bandwidth
```

```
## [1] 262.4867
```

```
m_pi = locpoly(x2, y,bandwidth=h_pi)

plot(x2, y)
lines(m_pi, col='red', lwd=2)
```



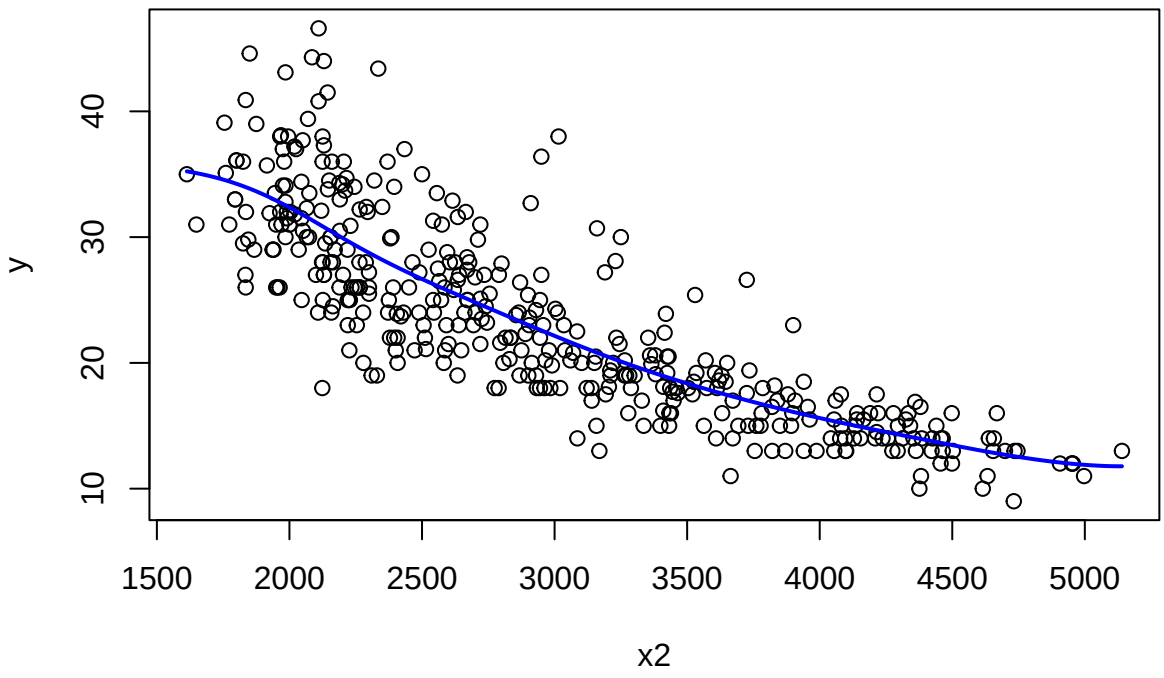

```
bw_vc = npregbw(y~x2, regtype="ll") #the degree must be indicated
```

```
## Multistart 1 of 1 |Multistart 1 of 1 |Multistart 1 of 1 |Multistart 1 of 1 /Multistart 1 of 1 |Multis
```

```
h_vc = bw_vc$bw #cross-validation bandwidth

m_vc = locpoly(x2,y, bandwidth = h_vc)

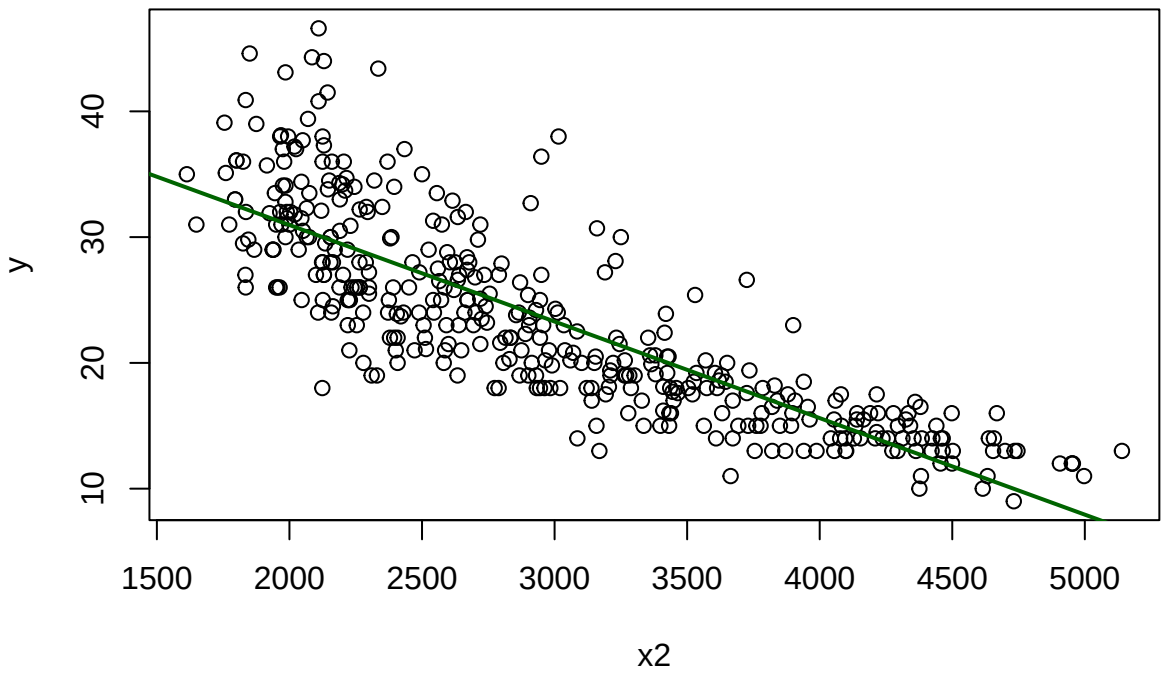
plot(x2, y)
lines(m_vc, col="blue", lwd=2)
```




```
lr = lm(y~x2)
lr
```

```
##
## Call:
## lm(formula = y ~ x2)
##
## Coefficients:
## (Intercept)           x2
##   46.317364    -0.007677
```

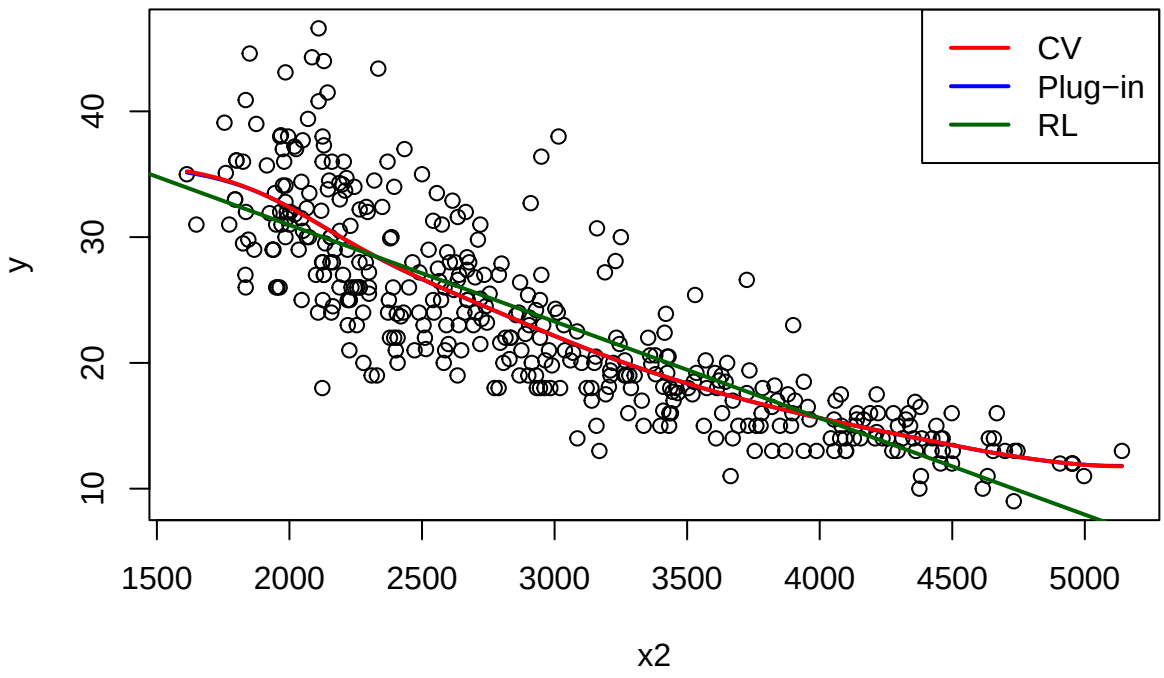
```
plot(x2, y)
abline(lr, col="darkgreen", lwd=2)
```



Its expression would be:

$y_{mask} = 46.317364 - 0.007677x_2$

```
plot(x2,y)
lines(m_pi,col='blue',lwd=2)
lines(m_vc,col='red',lwd=2)
abline(lr,col='darkgreen',lwd=2)
legend("topright", legend = c("CV", "Plug-in","RL"), col = c("red","blue","darkgreen"),
       lwd = 2)
```



In this case we see that CV and plug-in found a similar bandwidth, resulting in very similar estimates.

```
pred_pi <- approx(x = m_pi$x,y = m_pi$y,
                  xout = 3000)$y
pred_cv <- approx(x = m_vc$x,y = m_vc$y,
                  xout = 3000)$y
pred_reg <- predict(lr,newdata=data.frame(x2=3000))

predictions <- c(pred_pi,pred_cv,pred_reg)
names(predictions) <- c("Plug in","CrossValidation","RL")

predictions
```

```
##         Plug in CrossValidation             RL
##        22.14329        22.15018        23.28753
```